

C++中结构体作为函数参数的三种传递方式比较

安安这就给你讲明白！咱们把「结构体数组」和「结构体做函数参数（值传递 / 地址传递）」拆成三部分，用大白话 + 代码示例讲清楚，保证你能直接抄着用～

一、结构体基础回顾

结构体（`struct`）是 C++ 里用来把不同类型的数据打包成一个整体的工具，比如我们定义一个学生结构体：

代码块

```
1  #include <iostream>
2  #include <string>
3  using namespace std;
4
5  // 定义学生结构体
6  struct Student {
7      string name; // 姓名
8      int age;      // 年龄
9      double score; // 分数
10 };
```

二、结构体数组：批量存多个结构体

结构体数组，就是**元素类型为结构体的数组**，用来一次性管理多个相同类型的结构体数据，比如全班学生信息。

1. 定义与初始化

代码块

```
1  // 方式1：先定义，再赋值
2  Student stuArr[3]; // 定义能存3个Student的数组
3
4  stuArr[0].name = "张三";
5  stuArr[0].age = 18;
6  stuArr[0].score = 90.5;
7
```

```
8   stuArr[1].name = "李四";
9   stuArr[1].age = 19;
10  stuArr[1].score = 85.0;
11
12  stuArr[2].name = "王五";
13  stuArr[2].age = 18;
14  stuArr[2].score = 95.5;
15
16  // 方式2: 定义时直接初始化 (推荐)
17  Student stuArr[3] = {
18      {"张三", 18, 90.5},
19      {"李四", 19, 85.0},
20      {"王五", 18, 95.5}
21  };
```

2. 遍历结构体数组

可以用普通 for 循环，也可以用范围 for (C++11+)：

代码块

```
1  // 普通for循环
2  for(int i = 0; i < 3; i++) {
3      cout << "姓名: " << stuArr[i].name
4          << " 年龄: " << stuArr[i].age
5          << " 分数: " << stuArr[i].score << endl;
6  }
7
8  // 范围for循环 (更简洁)
9  for(const auto& s : stuArr) { // 用const&避免拷贝, 只读更安全高效
10     cout << "姓名: " << s.name
11         << " 年龄: " << s.age
12         << " 分数: " << s.score << endl;
13 }
```

三、结构体做函数参数：值传递 vs 地址传递

这是重点！核心区别就是：**传值会拷贝结构体，传地址 / 引用直接操作原结构体。**

1. 值传递 (Pass by Value)

原理

函数接收的是**结构体的一份拷贝**，函数里对参数的修改，只会影响拷贝的副本，不会影响原结构体。

代码示例

代码块

```
1 // 值传递：函数接收Student的拷贝
2 void printStudent(Student s) {
3     cout << "姓名: " << s.name << " 年龄: " << s.age << endl;
4     s.age = 99; // 修改的是拷贝的副本，原结构体不会变
5 }
6
7 int main() {
8     Student stu = {"张三", 18, 90.5};
9     printStudent(stu);
10    cout << "原结构体年龄: " << stu.age << endl; // 输出还是18，不是99
11    return 0;
12 }
```

优缺点

✅ 优点：安全，函数里的修改不会污染原数据

❌ 缺点：如果结构体很大（比如有很多成员），拷贝会浪费内存和时间，效率低

2. 地址传递（Pass by Address，也叫指针传递）

原理

函数接收的是**结构体的地址（指针）**，函数里通过指针直接操作原结构体的内存，修改会直接生效。

代码示例

代码块

```
1 // 地址传递：函数接收Student的指针
2 void modifyStudent(Student* s) {
3     // 指针访问成员要用 -> 运算符
4     s->age = 99;
5     s->score = 100.0;
6 }
7
8 int main() {
9     Student stu = {"张三", 18, 90.5};
10    modifyStudent(&stu); // 要传结构体的地址
11    cout << "修改后年龄: " << stu.age << " 分数: " << stu.score << endl;
12    // 输出：修改后年龄：99 分数：100
```

```
13     return 0;
14 }
```

拓展：结构体数组做函数参数（本质也是地址传递）

数组名本身就是首元素地址，所以把结构体数组传给函数，本质也是传地址：

代码块

```
1 // 接收结构体数组的函数
2 void printStudentArr(Student arr[], int n) {
3     // arr等价于Student* arr，函数里直接操作原数组
4     for(int i = 0; i < n; i++) {
5         cout << arr[i].name << " " << arr[i].age << endl;
6     }
7 }
8
9 int main() {
10     Student stuArr[2] = {"张三",18}, {"李四",19};
11     printStudentArr(stuArr, 2); // 直接传数组名
12     return 0;
13 }
```

3. 补充：引用传递（Pass by Reference, C++ 常用）

引用是 C++ 的特性，和地址传递效果一样，但语法更简单，也更安全（不会有空指针风险），推荐优先用。

代码示例

代码块

```
1 // 引用传递：函数接收Student的引用（相当于原结构体的别名）
2 void modifyStudentRef(Student& s) {
3     s.age = 99; // 直接修改原结构体，和原变量是同一个东西
4 }
5
6 // 只读场景用const引用，既高效又安全（不允许修改）
7 void printStudentRef(const Student& s) {
8     cout << s.name << " " << s.age << endl;
9     // s.age = 99; // 编译报错，const引用不允许修改
10 }
11
12 int main() {
```

```
13     Student stu = {"张三", 18, 90.5};
14     modifyStudentRef(stu); // 直接传变量名，不用加&
15     printStudentRef(stu);
16     return 0;
17 }
```

四、三种传递方式对比表

传递方式	语法示例	是否修改原数据	效率	适用场景
值传递	<code>void func(Student s)</code>	✗ 不能	低（拷贝开销）	结构体很小、且不需要修改数据时
指针传递	<code>void func(Student* s)</code>	✓ 能	高（直接操作原内存）	C 语言兼容场景，或需要显式表示“修改数据”时
引用传递	<code>void func(Student& s)</code>	✓ 能（const 引用不能）	高（无拷贝开销）	C++ 推荐，只读用 <code>const&</code> ，修改用普通引用

五、完整可运行示例（直接复制就能跑）

```
代码块
1  #include <iostream>
2  #include <string>
3  using namespace std;
4
5  struct Student {
6      string name;
7      int age;
8      double score;
9  };
10
11 // 值传递示例
12 void printByValue(Student s) {
13     cout << "[值传递] 姓名: " << s.name << " 年龄: " << s.age << endl;
14     s.age = 99; // 只修改副本
```

```

15 }
16
17 // 指针传递示例
18 void modifyByPointer(Student* s) {
19     s->age = 99;
20     s->score = 100.0;
21 }
22
23 // 引用传递示例
24 void modifyByRef(Student& s) {
25     s->age = 20;
26     s->score = 95.0;
27 }
28
29 // 结构体数组示例
30 void printArr(const Student arr[], int n) {
31     for(int i = 0; i < n; i++) {
32         cout << arr[i].name << " " << arr[i].age << " " << arr[i].score <<
endl;
33     }
34 }
35
36 int main() {
37     // 单个结构体测试
38     Student stu = {"张三", 18, 90.5};
39
40     printByValue(stu);
41     cout << "值传递后原年龄: " << stu.age << endl; // 还是18
42
43     modifyByPointer(&stu);
44     cout << "指针传递后年龄: " << stu.age << " 分数: " << stu.score << endl;
45
46     modifyByRef(stu);
47     cout << "引用传递后年龄: " << stu.age << " 分数: " << stu.score << endl;
48
49     // 结构体数组测试
50     Student stuArr[2] = {"李四", 19, 85.0}, {"王五", 18, 95.5};
51     cout << "\n结构体数组遍历: " << endl;
52     printArr(stuArr, 2);
53
54     return 0;
55 }

```

1. **结构体数组**就是元素为结构体的数组，用来批量管理同类型结构体数据，遍历和普通数组类似。
 2. **值传递**会拷贝结构体，修改不影响原数据，效率低，不推荐处理大结构体。
 3. **\\ 地址传递（指针 / 引用）** 直接操作原结构体，修改会生效，效率高，推荐用 `const&` 做只读操作，普通引用做修改操作。
-

要不要我给你出几道小练习题，帮你彻底搞懂这三种传递方式的区别？

（注：文档部分内容可能由 AI 生成）